# Praktikum 2
# Socket Programming

| | | |
|---|---|---|
| **NAMA** | : | Ihsanta Zaki Sanjaya |
| **KELAS** | : | 2 Teknik Komputer B |
| **NRP** | : | 3223600058 |
| **DOSEN** | : | Maretha Ruswiansari,S.ST., M.T |
| **MATA KULIAH** | : | Praktikum Jaringan Komputer |
| **TANGGAL** | : | Kamis, 27 Februari 2025 |

**D4 TEKNIK KOMPUTER**
**POLITEKNIK ELEKTRONIKA NEGERI SURABAYA**
**2025**

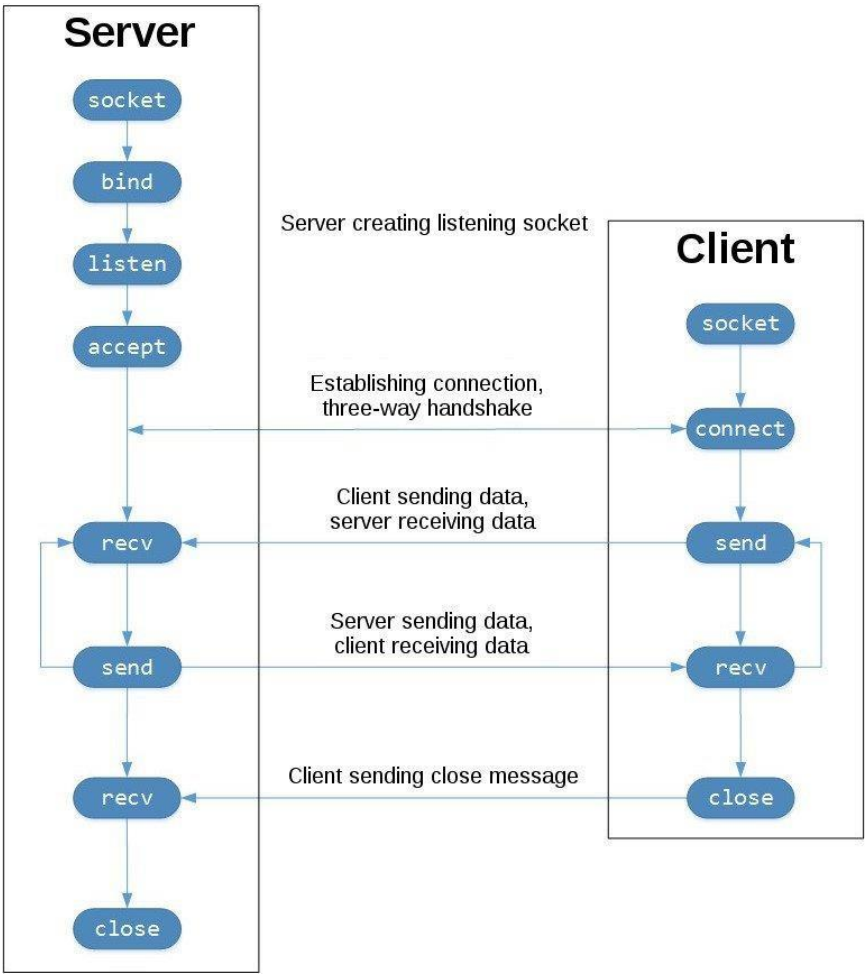
# Socket Programming
# TCP dan UDP Echo Client/Server
## Python

## I. Tujuan

1. Mahasiswa dapat menjelaskan konsep jaringan berbasis *client-server*
2. Mahasiswa dapat menjelaskan konsep pemrograman *socket* berbasis TCP dan UDP
3. Mahasiswa dapat menjelaskan cara kerja program TCP dan UDP *echo client/server*

## II. Pendahuluan

*Transmission Control Protocol* (TCP) merupakan protokol pada *transport layer* berbasis *connection-oriented*. TCP menjamin realibitas pengiriman data. Selain itu, TCP memiliki mekanisme *three-way handshake* sebelum *client-server* dapat saling mengirimkan data. Pada gambar 1 berikut menunjukkan alur komunikasi *client-server* mengggunakan TCP.



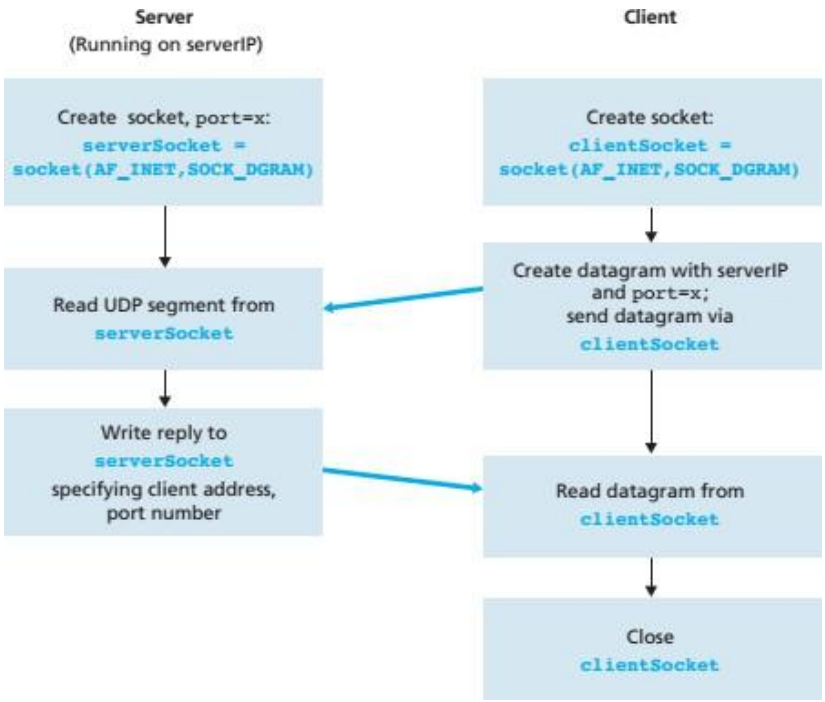

*Gambar 1.* TCP Socket

Pada gambar 1 menunjukkan bahwa di sisi *server* terdapat beberapa *socket* API yang digunakan agar dapat berkomunikasi pada jaringan. Socket API pada TCP server yang bertugas untuk membuat

*listening socket* yaitu : **`socket( ), bind( ), listen( ), accept( )`**. Proses *listening socket* merupakan saat dimana *server* menunggu koneksi dari *client.* Jadi, saat *client* menghubungkan ke server dengan **`connect( )`**, hal ini mengawali proses *connection establishment.* Kemudian *server* akan menerima koneksi dari *client* melalui **`accept( )`**, dan mekanisme *three-way handshake* selesai. Selanjutnya, *client* dan server dapat saling berkomunikasi atau bertukar data menggunakan **`send( )`** dan **`recv( )`**.

User Datagram Protocol (UDP) merupakan protokol yang bersifat *connectionless.* UDP tidak dapat menjamin pengiriman maupun urutan paket. Sealain itu juga tidak ada mekanisme *handshaking.* Sehingga, reliabilitas UDP tidak sebaik TCP. UDP menyediakan *checksum* untuk integritas data dan *port number* sebagai pengalamatan. UDP diperlukan untuk aplikasi yang membutuhkan komunikasi efisien dan cepat tanpa mempermasalahkan *packet loss*.

Pada gambar 2 menunjukkan proses pengiriman data untuk komunikasi menggunakan protokol UDP. Bila dilihat pada gambar ataupun pada prosedur percobaan, pada sisi server tidak ada proses *listen* dan *accept* seperti pada TCP. Namun, hanya ada proses *bind* untuk menghubungkan *socket* dengan *port* dan proses menunggu pesan yang dikirim dari *client.* Sedangkan pada sisi *client* menggunakan `sendto()` untuk mengirimkan pesan secara langsung ke *server,* dan `recvfrom()` untuk menerima pesan balasan dari *server*.




*Gambar 2.* UDP Socket

Pada praktikum kali ini, kita akan mempelajari *socket programming,* dimana program yang akan dibuat, memungkinkan *client* dan *server* berkomunikasi. Program *echo server* dan *echo client* menggunakan protokol TCP dan UDP. *Server* akan membalas atau mengulang kembali data yang dikirim oleh *client.*

## III. Prosedur Percobaan

### A. TCP

1. Buatlah program tcp *echo server.*

```python
#!/usr/bin/env python3

import socket
import sys
import argparse
host='localhost'
data_payload=2048
backlog=5
def echo_server(port):
    # Membuat socket object dengan TCP
    sock=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
    sock.setsockopt(socket.SOL_SOCKET,socket.SO_REUSEADDR,1)
    server_address=(host,port)
    print("Starting up echo server on %s port %s" % server_address)
    # Bind the socket, Menentukan port yang digunakan pada host tujuan
    sock.bind(server_address)
    # Listen for incoming connection from clients, Nilai backlog menentukan jumlah maksimum antrian koneksi
    sock.listen(backlog)
    while True:
        print("Waiting to receive message from client")
        # Establish a connection, menunggu koneksi dari client
        client,address=sock.accept()
        # Server menerima data dari client
        data=client.recv(data_payload)
        if data:
            print("Data: %s" %data.decode('utf-8'))
            #Server mengirim data ke client
            client.send(data)
            print("sent %s bytes back to %s" % (data.decode('utf-8'),address))
        # Koneksi berakhir
        client.close()
if __name__=='__main__':
    parser=argparse.ArgumentParser(description='Socket Server Example')
    parser.add_argument('--port',action="store",dest="port",type=int,required=True)
    given_args=parser.parse_args()
    port=given_args.port
    echo_server(port)
```

Hasil:

```python
#!/usr/bin/env python3
import socket
import sys
import argparse

host = '0.0.0.0'
data_payload = 2048
backlog = 5

def echo_server(port):
    # Create a socket object using TCP
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server_address = (host, port)
    print("Starting up echo server on %s port %s" % server_address)
    # Bind the socket, specifying the port to be used on the destination host
    sock.bind(server_address)
    # Listen for incoming connections from clients, the backlog value determines the ma
    sock.listen(backlog)
    while True:
        print("Waiting to receive message from client")
        # Establish a connection, waiting for connection from client
        client, address = sock.accept()
        # Server receives data from client
        data = client.recv(data_payload)
        if data:
            print("Data: %s" % data.decode('utf-8'))
            # Server sends data back to client
            client.send(data)
            print("sent %s bytes back to %s" % (data.decode('utf-8'), address))
        # Connection terminated
        client.close()

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Socket Server Example')
    parser.add_argument('--port', action="store", dest="port", type=int, required=True)
    given_args = parser.parse_args()
    port = given_args.port
    echo_server(port)
```

2. Buatlah program tcp *echo client.*

```python
#!/usr/bin/env python3

import socket
import sys
import argparse
host='localhost'
def echo_client(port):
    # Membuat socket object dengan TCP
    sock=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
    server_address=(host,port)
    print("Connecting to %s port %s" % server_address)
    # Mengawali koneksi dengan server
    sock.connect(server_address)
    try:
        # Mengirim data ke server
        message="Hello World!!! This will be echoed"
        print("Sending %s" % message)
        sock.sendall(message.encode('utf-8'))
        # Melihat respon
        amount_received=0
        amount_expected=len(message)
        while amount_received < amount_expected:
            # Menerima data, balasan dari server
            data=sock.recv(16)
            amount_received += len(data)
            print("Received: %s" %data.decode('utf-8'))
    except socket.error as e:
        print("Socket error: %s" %str(e))
    except Exception as e:
        print("Other exception %s" %str(e))
    finally:
        print("Closing connection to the server")
        sock.close()
if __name__=='__main__':
    parser=argparse.ArgumentParser(description='Socket Server Example')
    parser.add_argument('--port',action="store",dest="port",type=int,required=True)
    given_args=parser.parse_args()
    port=given_args.port
    echo_client(port)
```

Hasil:

```python
#!/usr/bin/env python3
import socket
import sys
import argparse
host='192.168.251.182'
def echo_client(port):
    #Membuat socket object dengan TCP
    sock=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_address=(host, port)
    print("Connecting to %s port %s" % server_address)
    # Mengawali koneksi dengan server
    sock.connect(server_address)
    try:
        # Mengirim data ke server
        message="Hello World!!! This will be echoed"
        print("Sending %s" % message)
        sock.sendall(message.encode('utf-8'))
        #Melihat respon
        amount_received=0
        amount_expected=len(message)
        while amount_received < amount_expected:
            # Menerima data, balasan dari server
            data=sock.recv(16)
            amount_received += len(data)
            print("Received: %s" %data.decode('utf-8'))
    except socket.error as e:
        print("Socket error: %s" %str(e))
    except Exception as e:
        print("Other exception %s" %str(e))
    finally:
        print("Closing connection to the server")
        sock.close()

if __name__ == '__main__':
    parser=argparse.ArgumentParser(description='Socket Server Example')
    given_args=parser.parse_args()
    port=given_args.port
    parser.add_argument('--port', action="store", dest="port", type=int, required=True)
    echo_client(port)
```

3. Jalankan program *echo server* terlebih dulu. Tampilkan hasil pengujian anda.

```
# python3 tcpechoserver.py --port=3456
Starting up echo server on localhost port 3456
Waiting to receive message from client
```

Catatan : port tersebut adalah port *server*, dimana *client* nantinya akan menggunakan port tersebut agar terhubung ke *server,* gunakan *non privileged ports* yaitu port > 1023

Hasil:

```
C:\Users\FoldExz\Desktop>python tcpechoserver.py --port=3456
Starting up echo server on 0.0.0.0 port 3456
Waiting to receive message from client
```

4. Jalankan program *echo client* secara terpisah *terminal* atau *command prompt* dari program *echo server*. Tampilkan hasil pengujian anda.

```
# python3 tcpechoclient.py --port=3456
Connecting to localhost port 3456
Sending Hello World!!! This will be echoed
Received: Hello World!!! T
Received: his will be echo
Received: ed
Closing connection to the server
```

Hasil:

```
(base) D:\sem 4\JArkom\wireshark>python tcpechoclient.py --port=3456
Connecting to 192.168.251.182 port 3456
Sending Hello World!!! This will be echoed
Received: Hello World!!! T
Received: his will be echo
Received: ed
Closing connection to the server
```

Sedangkan pada *server* akan tampil seperti berikut :

```
Data: Hello World!!! This will be echoed
sent Hello World!!! This will be echoed bytes back to ('127.0.0.1',
48486)
Waiting to receive message from client
```

Hasil:

```
Starting up echo server on 0.0.0.0 port 3456
Waiting to receive message from client
Data: Hello World!!! This will be echoed
sent Hello World!!! This will be echoed bytes back to ('192.168.251.103', 61152)
Waiting to receive message from client
```

5. Pada pengujian di prosedur 3 dan 4 menggunakan *localhost,* anda dapat juga menggunakan *host* yang berbeda dengan menyesuaikan *IP address* tujuan.
6. Buatlah program *echo client/server*, dimana anda bisa memberikan inputan melalui program *client* kemudian *server* akan membalas dengan mengirimkan ulang ke *client* apapun yang anda kirimkan sebelumnya. Program *client* dapat membaca inputan dari keyboard dan anda dapat mengirimkan data ke server beberapa kali selama koneksi dari *client* belum diakhiri.
7. Lampirkan program *echo client/server* yang anda buat untuk prosedur no. 6 pada laporan anda. Tampilkan hasil setelah anda menjalankan program yang anda buat tersebut.

Program server:

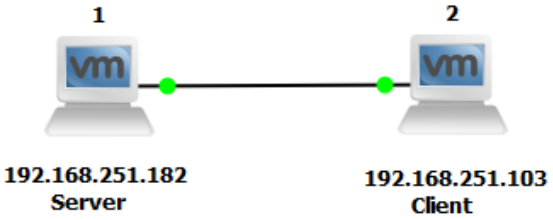
```python
import socket
import argparse
HOST = '0.0.0.0'
BUFFER_SIZE = 1024

def echo_server(port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server_address = (HOST, port)
    sock.bind(server_address)
    sock.listen(5)
    print(f"Server berjalan di {HOST} port {port}...")
    while True:
        client_socket, client_address = sock.accept()
        print(f"Terhubung dengan {client_address}")
        while True:
            data = client_socket.recv(BUFFER_SIZE)
            if not data:
                break
            message = data.decode('utf-8')
            print(f"Pesan diterima: {message}")

            if message.lower() == "exit":
                print("Client meminta untuk mengakhiri koneksi.")
                client_socket.sendall("Koneksi ditutup oleh server.".encode('utf-8'))
                break
            client_socket.sendall(data)
        client_socket.close()
        print("Koneksi ditutup.")
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Echo Server")
    parser.add_argument('--port', type=int, required=True, help="Port untuk server")
    args = parser.parse_args()
    echo_server(args.port)
```

Program client:

```python
import socket
import argparse
HOST = '192.168.251.182'
def echo_client(port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_address = (HOST, port)
    print(f"Menghubungkan ke {HOST} port {port}...")
    sock.connect(server_address)
    try:
        while True:
            message = input("Masukkan pesan: ")
            sock.sendall(message.encode('utf-8'))
            if message.lower() == "exit":
                print("Menutup koneksi ke server.")
                break
            data = sock.recv(1024)
            print(f"Server mengembalikan: {data.decode('utf-8')}")
    except Exception as e:
        print(f"Terjadi kesalahan: {e}")
    finally:
        sock.close()
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Echo Client")
    parser.add_argument('--port', type=int, required=True)
    args = parser.parse_args()
    echo_client(args.port)
```

Hasil:

Topologi




a. Sisi client

```
foldexz@fxz:~$ python3 clientt.py --port=3456
Menghubungkan ke 192.168.251.182 port 3456...
Masukkan pesan: Tugas JarKom kelompok 10
Server mengembalikan: Tugas JarKom kelompok 10
Masukkan pesan: Ihsanta Zaki 3223600058
Server mengembalikan: Ihsanta Zaki 3223600058
Masukkan pesan: Labib Al Afaf 3223600050
Server mengembalikan: Labib Al Afaf 3223600050
Masukkan pesan: exit
Menutup koneksi ke server
```

b. Sisi server

```
foldexz@fxz:~$ python3 servert.py --port=3456
Starting up echo server on 0.0.0.0 port 3456
Server berjalan di 0.0.0.0 port 3456...
Terhubung dengan ('192.168.251.103', 60216)
Pesan diterima: Tugas JarKom kelompok 10
Pesan diterima: Ihsanta Zaki 3223600058
Pesan diterima: Labib Al Afaf 3223600050
Pesan diterima: exit
Client meminta untuk mengakhiri koneksi
Koneksi ditutup
```

8. Analisa program *echo client/server* pada prosedur 1 dan 2 serta pada tugas di prosedur 6. Jelaskan bagaimana penerapan mekanisme TCP pada *socket programming* dan perintah-perintah yang digunakan pada program *socket* yang telah dibuat.

Analisis:

Penerapan mekanisme TCP pada program-program tersebut dapat terlihat dari penggunaan socket.AF_INET untuk IPv4 dan socket.SOCK_STREAM untuk protokol TCP, yang menjamin pengiriman data secara berurutan dan andal. Pada sisi server, fungsi bind() digunakan untuk menetapkan alamat IP dan port tertentu, sementara listen() memungkinkan server menunggu koneksi masuk dengan antrean tertentu. Ketika ada client yang terhubung, server menerima koneksi menggunakan accept(), lalu membaca data yang dikirimkan client dengan recv(). Setelah diproses, server mengembalikan data menggunakan sendall(), mencerminkan prinsip TCP yang memastikan data dikirim dan diterima secara utuh. Jika client ingin mengakhiri koneksi, pesan "exit" dikirim dan server akan menutup koneksi dengan close().

Di sisi client, mekanisme TCP diawali dengan pembuatan socket menggunakan socket.socket(), kemudian koneksi dibuka ke server menggunakan connect(). Client mengirim pesan melalui sendall(), yang memastikan seluruh data terkirim tanpa terputus. Setelah itu, client menunggu respons server menggunakan recv(), memastikan data diterima dengan lengkap. Dalam program clientt.py dan servert.py, komunikasi terjadi dalam bentuk echo, di mana server mengembalikan data yang diterima dari client. Sementara pada program di awal mekanisnya serupa namun lebih sederhana.

B. UDP

1. Buatlah program udp *echo server.*

```python
#!/usr/bin/env python3

import socket
import sys
import argparse

host = 'localhost'
data_payload = 2048

def echo_server(port):
    # Membuat socket object menggunakan UDP
    sock = socket.socket (socket.AF_INET, socket.SOCK_DGRAM)
    server_address = (host, port)
    print("Starting up echo server on %s port %s" % server_address)
    # Bind the socket, menentukan port yang digunakan pada host tujuan
    sock.bind(server_address)

    while True:
        # Menunggu menerima pesan dari client
        print("Waiting to receive message from client")
        data, address = sock.recvfrom(data_payload)
        print("received %s bytes from %s" % (len(data), address))
        print("Data: %s" %data.decode('utf-8'))
        if data:
            # Server mengirim pesan ke client
            sent = sock.sendto(data, address)
            print("sent %s bytes back to %s" % (sent, address))

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Socket Server Example')
    parser.add_argument('--port', action="store", dest="port", type=int, required=True)
    given_args = parser.parse_args()
    port = given_args.port
    echo_server(port)
```

Hasil:

```python
#!/usr/bin/env python3
import socket
import sys
import argparse

host = '0.0.0.0'
data_payload = 2048

def echo_server(port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server_address = (host, port)
    print("Starting up echo server on %s port %s" % server_address)
    sock.bind(server_address)

    while True:
        print("Waiting to receive message from client")
        data, address = sock.recvfrom(data_payload)
        print("Received %s bytes from %s" % (len(data), address))
        print("Data: %s" % data.decode('utf-8'))
        if data:
            sent = sock.sendto(data, address)
            print("Sent %s bytes back to %s" % (sent, address))

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Socket server example')
    parser.add_argument('--port', action="store", dest="port", type=int, required=True)
    given_args = parser.parse_args()
    port = given_args.port
    echo_server(port)
```

2. Buatlah program *echo client.*

```python
#!/usr/bin/env python3

import socket
import sys
import argparse

host = 'localhost'
data_payload = 2048

def echo_client(port):
    # Membuat socket object menggunakan UDP
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server_address = (host, port)
    print("Connecting to %s port %s" % server_address)
    message = 'This is the message.  It will be repeated.'
    print()
    try:
        # Mengirim pesan ke server
        message = "Good Morning!!! This will be echoed"
        print ("Sending %s" % message)
        sent = sock.sendto(message.encode('utf-8'), server_address)
        # Menerima balasan dari server
        data, server = sock.recvfrom(data_payload)
        print("received %s" % data.decode('utf-8'))
    finally:
        print("Closing connection to the server")
        sock.close()
if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Socket Server Example')
    parser.add_argument('--port', action="store", dest="port", type=int,required=True)
    given_args = parser.parse_args()
    port = given_args.port
    echo_client(port)
```

Hasil:

```python
#!/usr/bin/env python3
import socket
import sys
import argparse

host = '192.168.251.182'
data_payload = 2048

def echo_client(port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server_address = (host, port)
    print("connecting to %s port %s" % server_address)
    message = 'this is message, it will be repeated.'
    print()
    try:
        message = 'hello! this will be echoed'
        print ("sending %s" % message)
        sent = sock.sendto(message.encode('utf-8'), server_address)
        data, server = sock.recvfrom(data_payload)
        print("received %s" % data.decode('utf-8'))
    finally:
        print("closing connection to the server")
        sock.close()
if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='socket server example')
    parser.add_argument('--port', action="store", dest="port", type=int, required=True)
    given_args = parser.parse_args()
    port = given_args.port
    echo_client(port)
```

3. Jalankan program *echo server* terlebih dulu. Tampilkan hasil pengujian anda.

```
# python3 udpechoserver.py --port=8765
Starting up echo server on localhost
port 8765 Waiting to receive message
from client
```

Hasil:

```
C:\Users\FoldExz\Desktop>python udpechoserver.py --port=8765
Starting up echo server on 0.0.0.0 port 8765
Waiting to receive message from client
```

4. Jalankan program *echo client* secara terpisah *terminal* atau *command prompt* dari program *echo server*. Tampilkan hasil pengujian anda.

```
# python3 udpechoclient.py --port=8765
Connecting to localhost port 8765
Sending Good Morning!!! This will be
echoed received Good Morning!!! This
will be echoed Closing connection to
the server
```

Hasil:

```
(base) D:\sem 4\JArkom\wireshark>python udpechoclient.py --port=8765
connecting to 192.168.251.182 port 8765

sending hello! this will be echoed
received hello! this will be echoed
closing connection to the server
```

Sedangkan pada *server* akan tampil seperti berikut :

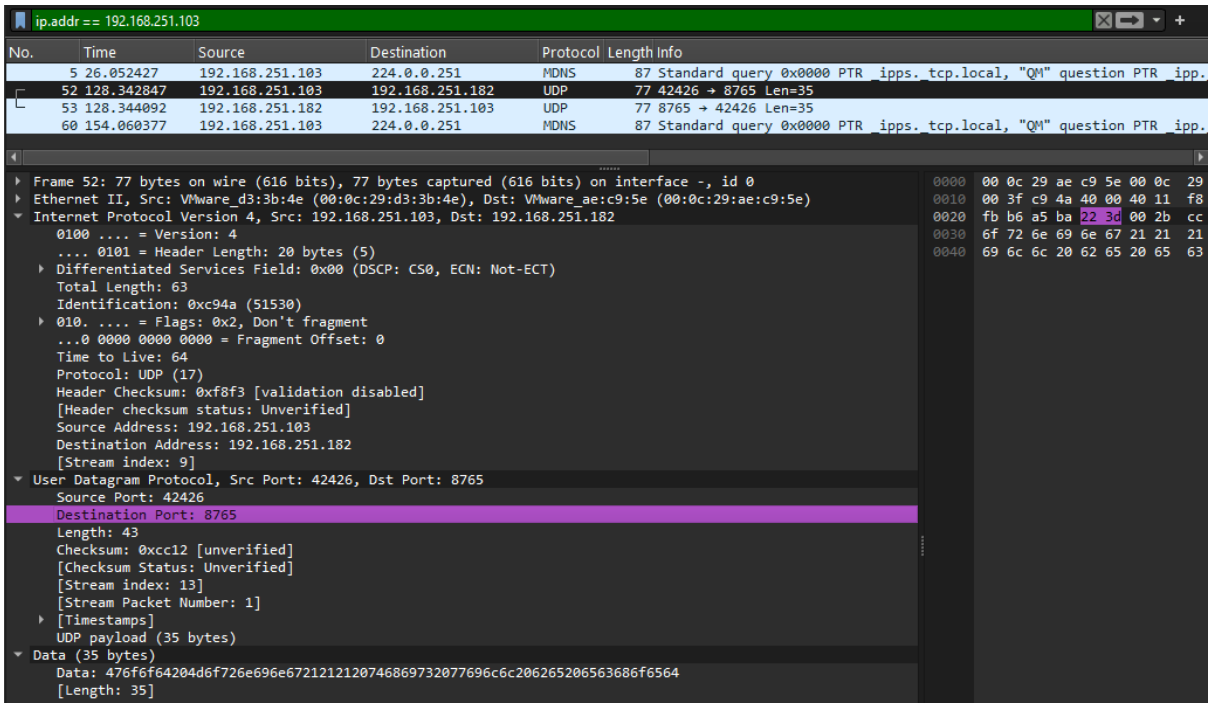
```
received 35 bytes from ('127.0.0.1',
55559) Data: Good Morning!!! This
will be echoed sent 35 bytes back to
('127.0.0.1', 55559)
```

Hasil:

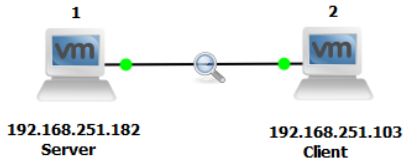
```
Starting up echo server on 0.0.0.0 port 8765
Waiting to receive message from client
Received 26 bytes from ('192.168.251.103', 56051)
Data: hello! this will be echoed
Sent 26 bytes back to ('192.168.251.103', 56051)
Waiting to receive message from client
Received 26 bytes from ('192.168.251.103', 50278)
Data: hello! this will be echoed
Sent 26 bytes back to ('192.168.251.103', 50278)
Waiting to receive message from client
```

5. Pada pengujian di prosedur 3 dan 4 menggunakan *localhost,* anda dapat juga menggunakan *host* yang berbeda dengan menyesuaikan *IP address* tujuan.
6. Lakukan *packet analysis* menggunakan aplikasi *Wireshark* untuk mengamati pengiriman data menggunakan protokol UDP

Hasil:

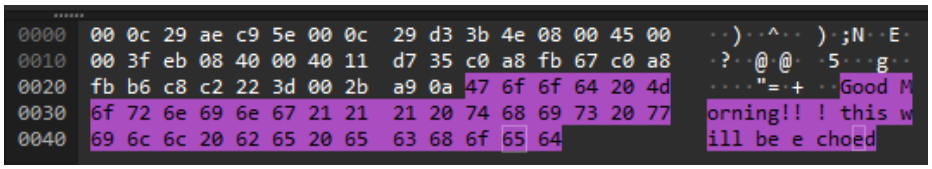



Catatan: Pengujian packet analysis menggunakan device yang berbeda namun dengan pengaturan IP yang sama.




Analisis:

Komunikasi antara client dan server menggunakan protokol UDP berlangsung secara singkat dengan adanya dua paket yang dipertukarkan. Pada frame 52, client (192.168.251.103) mengirimkan paket UDP ke server (192.168.251.182) melalui port tujuan 8765. Karena paket ini berasal dari client, paket ini memiliki source port 42426 yang merupakan ephemeral port. Sedangkan pada frame 53, server merespons dengan mengirimkan paket balasan melalui port asalnya, yaitu 8765, kembali ke ephemeral port 4246 milik client.

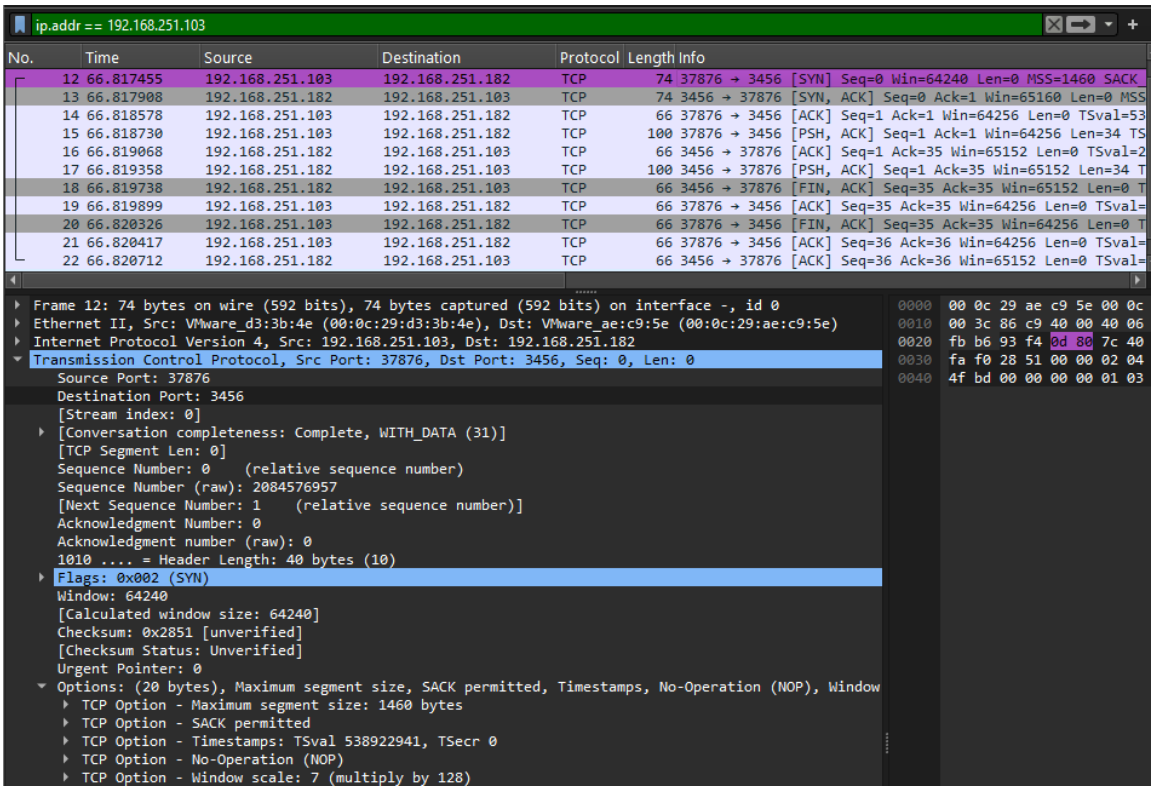



Pola komunikasi ini menunjukkan bahwa komunikasi UDP ini adalah one-time transaction, di mana tidak ada paket tambahan yang dikirim setelah respons diterima, sehingga komunikasi selesai dengan cepat. Ini menggambarkan salah satu karakteristik UDP yaitu connectionless, di mana data dikirim tanpa proses handshake, sehingga lebih cepat tetapi tidak menjamin keandalan pengiriman seperti TCP.

Pada UDP juga terlihat, bahwa pesan dari echonya dapat terlihat dan diamati melalui Wireshark karena UDP tidak memiliki mekanisme keamanan bawaan. Berbeda dengan TCP, yang lebih terenkripsi dan andal dalam menjaga integritas serta keamanan data selama transmisi.

7. Lakukan *packet analysis* menggunakan aplikasi *Wireshark* untuk mengamati pengiriman data menggunakan protokol TCP, gunakan program TCP socket pada praktikum sebelumnya

Hasil:




Analisis:

Komunikasi antara client (192.168.251.103) dan server (192.168.251.182, port 3456) menggunakan protokol TCP terdiri dari beberapa paket yang menunjukkan proses koneksi dan pertukaran data. Pada Frame 12, terlihat bahwa client menginisiasi koneksi dengan mengirimkan paket SYN dari port ephemeral 37876 ke port 3456 milik server. Server kemudian merespons dengan SYN-ACK pada Frame 13, yang menunjukkan bahwa server menerima permintaan koneksi. Client mengonfirmasi koneksi dengan mengirimkan ACK pada Frame 14, sehingga three-way handshake berhasil dilakukan, dan koneksi TCP resmi terjalin.

Setelah koneksi terbentuk, terlihat adanya transfer data dari client ke server menggunakan PSH, ACK pada Frame 15 dan 16, yang menandakan client mengirimkan payload ke server. Server kemudian membalas dengan paket ACK, mengonfirmasi penerimaan data. Pada Frame 19 dan 20, client mulai menutup koneksi dengan mengirimkan FIN, ACK, yang kemudian direspons oleh server dengan FIN, ACK pada Frame 21, diikuti dengan ACK terakhir dari client di Frame 22, menandakan bahwa koneksi TCP telah ditutup secara bersih. Proses ini menunjukkan sifat reliable dan connection-oriented dari TCP, di mana setiap tahap komunikasi dipastikan diterima sebelum melanjutkan ke tahap berikutnya.

8. Lakukan analisa dari pengamatan yang sudah anda lakukan menggunakan protokol TCP dan UDP. Jelaskan apa saja yang dapat anda analisa dari *capture* paket pada *Wireshark*, dan apa saja perbedaan pada paket data yang dikirimkan menggunaakan TCP dan UDP.

Analisis:

Pengiriman data dalam komunikasi dengan protokol UDP lebih cepat dibandingkan TCP, karena tanpa adanya mekanisme handshake atau pengaturan koneksi diawal. Pada pengamatan paket, terlihat bahwa client langsung mengirimkan data ke server menggunakan ephemeral port dan server merespons dengan paket balasan ke port asal client. Proses ini berlangsung singkat dan tidak ada mekanisme untuk memastikan apakah data benar-benar diterima dengan utuh atau tidak, sehingga UDP lebih cocok untuk aplikasi yang membutuhkan kecepatan tinggi dan toleransi terhadap kehilangan paket, seperti streaming video atau komunikasi VoIP.

Sebaliknya, komunikasi menggunakan protokol TCP memperlihatkan adanya proses three-way handshake sebelum data benar-benar dikirim. Pada analisis paket, terlihat bahwa client menginisiasi koneksi dengan mengirimkan SYN, diikuti oleh respons SYN-ACK dari server, dan dikonfirmasi dengan ACK dari client. Setelah koneksi terbentuk, data dikirimkan dengan mekanisme pengakuan (ACK) untuk memastikan keandalan pengiriman. Saat koneksi berakhir, terjadi proses terminasi menggunakan FIN-ACK, yang memastikan bahwa kedua pihak telah sepakat untuk menutup koneksi. Hal ini menunjukkan bahwa TCP lebih andal karena menjamin urutan dan integritas data, sehingga lebih cocok digunakan untuk transfer file, email, dan browsing web.

Perbedaan utama antara UDP dan TCP dalam analisis paket terletak pada mekanisme pengiriman dan kontrol koneksi. UDP bersifat connectionless, tidak memiliki fitur error checking yang kompleks, serta tidak menjamin urutan paket yang diterima. Sementara itu, TCP memiliki mekanisme kontrol yang lebih ketat, termasuk flow control, error checking, dan retransmisi paket yang hilang. Hal ini menyebabkan TCP lebih lambat dibandingkan UDP, tetapi lebih dapat diandalkan dalam pengiriman data. Oleh karena itu, pemilihan protokol bergantung pada kebutuhan aplikasi, apakah lebih mengutamakan kecepatan atau keandalan dalam pengirimannya.

## Tugas Pertanyaan

1. Jelaskan beberapa keunggulan protokol TCP.

   Jawab:

   Reliability, menjadi keunggulan utama pada protokol TCP untuk komunikasi data yang membutuhkan keandalan tinggi. Dengan adanya mekanisme acknowledgment dan retransmisi jika adanya paket yang hilang, memastikan bahwa semua paket data sampai dengan benar hingga ke tujuan. TCP juga memiliki flow control yang mencegah pengirim mengirimkan data lebih cepat daripada yang bisa diterima oleh penerima, sehingga mencegah terjadinya kehilangan data akibat overload. Ketika data diterima disisi penerima, TCP mendukung pengurutan paket (sequencing) sehingga data yang dikirim akan diurutkan dalam urutan yang sesuai. Keamanannya dalam transmisi data juga membuatnya lebih unggul dibanding UDP, karena TCP membangun koneksi melalui three-way handshake sebelum mentransmisikan datanya.

2. Jelaskan *server socket method* dan *client socket method* pada *socket programming* mengggunakan Python.

   Jawab:

   Server socket methods mencakup socket() untuk membuat socket, bind() untuk mengikat socket ke alamat dan port tertentu, listen() untuk menunggu koneksi dari client, dan accept() untuk menerima koneksi yang masuk. Setelah terhubung, server menggunakan recv() untuk menerima data dari client dan send() untuk mengirimkan data kembali. Di sisi client, client socket methods meliputi socket() untuk membuat socket, connect() untuk menghubungkan ke server, sendall() untuk mengirim data, dan recv() untuk menerima balasan dari server. Sebagai contoh seperti dalam praktikum, dalam implementasinya client mengirim pesan "Hello World!!! This will be echoed" ke server dengan sendall(), dan server menerimanya

menggunakan recv(), kemudian mengembalikan pesan yang sama ke client menggunakan send(). Client kemudian menerima pesan tersebut secara bertahap dalam beberapa bagian menggunakan recv(), seperti yang terlihat dalam output client: "Received: Hello World!!! T", "Received: his will be echo", dan "Received: ed". Setelah komunikasi selesai, baik server maupun client menggunakan close() untuk menutup koneksi.

3. Jelaskan yang anda pahami mengenai mekanisme *three-way handshake.*
   Jawab:
   Three-way handshake adalah proses pada protokol TCP dalam membangun koneksi antara 2 perangkat sebelum bertukar informasi, yang biasanya terjadi pada client dan server. Sesuai namanya, proses ini terdiri dari 3 langkah utama:
   a. SYN (Synchronize), Client mengirim paket berisi informasi bahwa client ingin memulai komunikasi dengan server
   b. SYN-ACK (Synchronize-Acknowledge), Server menerima paket SYN dan mereply dengan paket SYN-ACK yang berisi konfirmasi penerimaan paket SYN sebelumnya.
   c. ACK (Acknowledge), Client menerima paket SYN-ACK dan mereply dengan ACK yang berisi bahwa client telah menerima balasan dari server, kemudian koneksi akan terbentuk.

   Contohnya dapat dilihat dalam komunikasi antara TCP client dan server pada praktikum, di mana client menghubungi server, lalu server merespons, dan akhirnya client memberikan konfirmasi sebelum mulai mengirimkan data sesungguhnya.

4. Jelaskan beberapa keunggulan protokol UDP.
   Jawab:
   Kecepatan transmisi, menjadi keunggulan utama pada protokol UDP untuk aplikasi yang membutuhkan kecepatan tinggi dan komunikasi real-time. Ini disebabkan UDP tidak memerlukan proses handshake, ack, atau retransmisi paket yang hilang, sehingga data dikirim dengan latensi yang rendah. Karena ukuran headernya tetap (8 byte) dan lebih ringan dibanding TCP, UDP memiliki overhead yang lebih kecil sehingga efisien dalam penggunaan bandwidth. Fitur broadcast dan multicast juga menjadikannya lebih unggul sehingga memungkinkan pengiriman ke banyak penerima sekaligus. Kesederhanaannya membuat UDP dapat digunakan dalam aplikasi dengan kebutuhan komunikasi cepat dan toleransi terhadap sedikit kehilangan data.

5. Sebutkan beberapa contoh penerapan protokol UDP pada pengiriman data di jaringan internet, serta jelaskan mengapa aplikasi tersebut menggunakan UDP.
   Jawab:
   Contoh penerapan protokol UDP adalah streaming media, game berbasis real-time multiplayer, dan voice over IP (VoIP). UDP dibutuhkan dalam berbagai aplikasi yang membutuhkan kecepatan tinggi dan toleransi terhadap kehilangan data. Pada media streaming seperti YouTube atau Netflix, sedikit kehilangan paket tidak terlalu berdampak karena video dan audio tetap bisa diputar tanpa gangguan yang signifikan. Dalam game multiplayer real-time, seperti Valorant, League of Legend, dan DOTA 2, karena kecepatan dan latensi (delay) sangat penting, kehilangan sedikit paket data tidak mempengaruhi gameplay secara keseluruhan. Sedangkan pada VoIP, seperti WhatsApp, Zoom, dan Google Meet, UDP diperlukan agar suara dikirim secepat mungkin tanpa menunggu retransmisi paket yang hilang, karena keterlambatan akan mengganggu komunikasi.

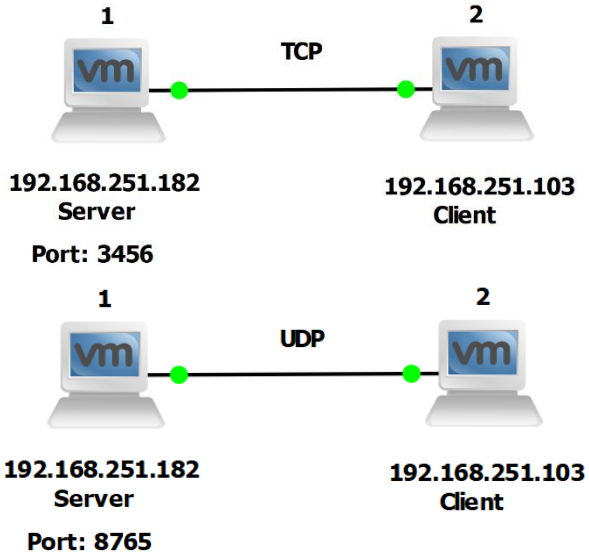
6. Jelaskan perbedaan TCP dan UDP.
   Jawab:

| | **Transmission Control Protocol (TCP)** | **User Datagram Protocol (UDP)** |
|---|---|---|
| **Jenis Layanan** | TCP adalah protokol berbasis koneksi, yang berarti perangkat komunikasi harus membangun koneksi sebelum mentransmisikan data dan menutup koneksi setelah selesai. | UDP adalah protokol berbasis datagram, tanpa overhead untuk membuka, mempertahankan, atau menutup koneksi. UDP lebih efisien untuk transmisi jaringan broadcast dan multicast. |
| **Keandalan** | TCP dapat diandalkan karena menjamin pengiriman data ke tujuan. | UDP tidak menjamin pengiriman data ke tujuan. |
| **Error checking mechanism** | TCP memiliki mekanisme pengecekan kesalahan yang lebih kompleks dengan flow control dan acknowledgment data. | UDP hanya memiliki mekanisme pengecekan kesalahan dasar menggunakan checksum. |
| **Acknowledge (Pengakuan Data)** | Terdapat segmen acknowledgment. | Tidak ada segmen acknowledgment. |
| **Urutan Data (Sequencing)** | TCP menjamin bahwa data sampai ke penerima dalam urutan yang benar. | UDP tidak menjamin urutan data. Jika urutan diperlukan, harus dikelola oleh lapisan aplikasi. |
| **Kecepatan** | TCP lebih lambat dibandingkan UDP. | UDP lebih cepat, lebih sederhana, dan lebih efisien daripada TCP. |
| **Retransmisi** | TCP dapat mengirim ulang paket yang hilang. | UDP tidak memiliki mekanisme retransmisi paket yang hilang. |
| **Ukuran Header** | Header TCP memiliki panjang variabel (20-60 byte). | Header UDP memiliki panjang tetap (8 byte). |
| **Bobot (Weight)** | TCP adalah protokol yang lebih berat karena memiliki lebih banyak fitur pengelolaan koneksi. | UDP lebih ringan dibandingkan TCP. |
| **Teknik Handshaking** | TCP menggunakan teknik handshake (SYN, ACK, SYN-ACK) sebelum memulai komunikasi. | UDP adalah protokol tanpa koneksi, sehingga tidak memerlukan handshake. |
| **Broadcasting** | TCP tidak mendukung broadcasting. | UDP mendukung broadcasting. |
| **Protocol** | TCP digunakan oleh HTTP, HTTPS, FTP, SMTP, dan Telnet. | UDP digunakan oleh DNS, DHCP, TFTP, SNMP, RIP, dan VoIP. |
| **Jenis Stream** | TCP mengirimkan data dalam bentuk byte stream. | UDP mengirimkan data dalam bentuk message stream. |
| **Overhead** | Overhead TCP lebih tinggi dibandingkan UDP. | Overhead UDP sangat rendah. |
| **Applications** | Digunakan dalam aplikasi yang membutuhkan komunikasi yang aman dan dapat diandalkan, seperti email, browsing web, dan layanan militer. | Digunakan dalam aplikasi yang membutuhkan komunikasi cepat tetapi tidak memerlukan keandalan tinggi, seperti VoIP, game online, dan streaming video/musik. |

Perbedaan utama antara TCP dan UDP terletak pada keandalan dan kecepatan. TCP lebih lambat karena memiliki mekanisme handshake, acknowledgment, dan retransmisi untuk memastikan data sampai dengan benar, sehingga cocok untuk aplikasi yang membutuhkan keandalan tinggi. Sebaliknya, UDP lebih cepat karena tidak memiliki proses tersebut, tetapi kurang andal dalam pengiriman data, sehingga lebih sesuai untuk aplikasi yang membutuhkan kecepatan tanpa terlalu mengkhawatirkan kehilangan paket.

## IV. Analisis

Dalam praktikum ini, kami membuat dan menguji program socket programming dengan menggunakan protokol TCP dan UDP. Program tersebut diuji dengan menggunakan dua device yang berbeda, masing-masing sebagai server dan client.




Di sisi server, alamat IP dan port ditetapkan secara statis (pada praktikum port 3456 untuk TCP dan port 8765 untuk UDP), sehingga server selalu siap menerima koneksi masuk. Sementara itu, di sisi client, port yang digunakan adalah ephemeral port, yaitu port sementara yang dipilih secara otomatis oleh sistem operasi untuk setiap koneksi keluar, sehingga nilainya bisa berubah-ubah pada setiap sesi komunikasi. Di mana server dan client menggunakan socket API khusus untuk membangun koneksi diantaranya. Proses di sisi server dimulai dengan pembuatan socket melalui fungsi socket(), diikuti dengan bind() untuk mengikat alamat IP dan port, serta listen() untuk menunggu koneksi masuk. Ketika client melakukan connect(), server menerima koneksi dengan menggunakan accept(), dan setelah itu terjadi pertukaran data menggunakan sendall() dan recv() sebelum koneksi ditutup dengan close(). Memastikan bahwa setiap paket data dikirim dan diterima dengan benar.

Pada penerapan TCP echo client/server, dengan adanya proses three-way handshake (SYN, SYN-ACK, dan ACK), ini menjamin bahwa kedua belah pihak siap untuk bertukar data, sehingga jika terjadi kehilangan paket atau error, mekanisme retransmisi dan acknowledgment memastikan data sampai dengan utuh. Selanjutnya, penggunaan fungsi sendall() memastikan bahwa seluruh pesan dikirim secara menyeluruh, sementara recv() membantu client menerima data dalam bagian-bagian kecil yang kemudian dirakit kembali, ini adalah mekanisme flow control dan error checking yang menjadi karakteristik dari TCP.

Pada penerapan UDP echo client/server, berbalik dibandingkan TCP. Karena UDP merupakan protokol connectionless, protokol ini tidak melakukan proses handshake sehingga pengiriman data berlangsung dengan latensi yang rendah namun tanpa jaminan keandalan. Pada program UDP echo client/server, server hanya perlu menggunakan bind() untuk mengikat socket ke port dan menunggu pesan dari client, sedangkan client menggunakan sendto() dan recvfrom() untuk komunikasi langsung. Analisis packet capture dengan Wireshark juga menunjukkan bahwa komunikasi UDP hanya melibatkan dua paket pertukaran data tanpa proses handshake, sehingga lebih cepat namun dengan adanya kemungkinan kehilangan paket. Sehingga membuat UDP lebih cocok untuk aplikasi yang memprioritaskan kecepatan seperti streaming media, game real-time, atau VoIP.

## V. Kesimpulan

Penerapan protokol TCP dan UDP dalam socket programming memiliki peran penting dalam mendukung berbagai aplikasi dengan kebutuhan yang berbeda. TCP menjamin pengiriman data secara terurut dan aman melalui mekanisme three-way handshake, yang memastikan koneksi stabil dan memungkinkan retransmisi data jika terjadi error atau hilang, sehingga cocok untuk aplikasi yang memerlukan keandalan tinggi seperti transfer file atau browsing web. Sebaliknya, UDP menawarkan komunikasi dengan latensi rendah dan overhead minimal karena bersifat connectionless, sehingga ideal untuk aplikasi real-time seperti streaming media, game online, dan VoIP meskipun tidak menjamin keutuhan data. Selain itu, penggunaan ephemeral port pada sisi client memungkinkan sistem operasi memilih port secara dinamis untuk setiap koneksi keluar, sehingga menghindari konflik port dan mendukung efisiensi dalam pengelolaan koneksi antara client-server.

## VI. Lampiran Laporan Sementara

Kelompok 10
Labib Al Afif 3223600050
Ihsanta Zaki S 3223600058

Laporan Sementara
Socket Programming TCP/IP

6/3 25

① Pengujian TCP echo server

| | X13 Thinkpad | MSI |
|---|---|---|
| Client/server | Server | Client |
| IP Address | 192.168.251.182 | 192.168.251.103 |
| Port number | 3456 ✓ | WiFi: 611522 Ethernet: 65509 |

② Pengujian UDP echo server

| | X13 Thinkpad | MSI |
|---|---|---|
| Client/server | Server | Client |
| IP Address | 192.168.251.182 | 192.168.251.103 |
| Port Number | 8765 ✓ | WiFi: 51175 Ethernet: 56051 |

Pengujian Pengiriman data menggunakan UDP echo server

- Tethering / WiFi

Client : Connecting to 192.168.251.182 port 8765 ✓
sending hello! this will be echoed
received hello! this will be echoed
closing connecting to the server

Server : Starting up echo server on 0.0.0.0 port 8765 ✓
Waiting to receive message from client
Received 26 bytes from ('192.168.251.103', 51175)
Data : hello! this will be echoed
Sent 26 bytes back to ('192.168.251.103', 51175)
Waiting to receive messag from client

- Ethernet

Client : connecting to 192.168.251.182 port 8765 ✓
sending hello! this will be echoed
received hallo! this will be echoed
closing connection to the server

Server : Starting up echo server on 0.0.0.0 port 8765 ✓
Waiting to receive message from client
Received 26 bytes from ('192.168.251.103', 56051)
Data : hello! this will be echoed
Sent 26 bytes back to ('192.168.251.103', 56051)

Pengujian Pengiriman data menggunakan TCP echo server

* Tethering

- Client : Connecting to 192.168.241.132 port 3456.
  Sending Hello World!!! This will be echoed
  Received : Hello World!!! T
  Received : this will be echo
  Received : ed
  Closing connection to the server

- Server : Starting up echo server on 0.0.0.0 port 3456
  Waiting to receive message from client
  Data : Hello World!!! This will be echoed
  sent : Hello World!!! This will be echoed bytes back to ('192.168.241.103', 61522)
  waiting to receive message from client

* Ethernet : Connecting to 192.168.241.132 port 3456

Client
  Sending Hello world!!! This will be echoed
  Received : Hello World!!! T
  Received : his will be echo
  Received : ed
  Closing connection to the server

Server : Starting up echo server on 0.0.0.0 port 3456
  Waiting to receive message from client
  Data : Hello world!!! This will be echoed
  Sent : Hello world!!! This will be echoed bytes back to (192.168.241.103', 65509 )
  Waiting to receive message from client